Application de gestion de tâches — Projet SI-SOR

Développé par Liutwin Piernet & Kévin Tambon

Technologies utilisées

Technologie	Rôle
React	Frontend SPA — routing, drag & drop
Deno / Oak / Vite	Backend n°1 — authentication JWT, proxy, WebSocket
Spring Boot / JPA	Backend n°2 — API REST métier, accès aux bases de données
MariaDB	Utilisateurs, tableaux, colonnes, tâches, participation, journaux (et token pour mot de passe oublié)
MongoDB	Commentaires et pièces jointes
nginx	Reverse proxy
Docker	Conteneurisation
GitLab	Versionnement du code source

Fonctionnalités mises en œuvre

— Partie publique
- ☑ Création de compte utilisateur
- ☑ Authentification
- ☑ Consultation des tableaux auxquels l'utilisateur participe
- ☑ Création d'un tableau
- ☑ Invitation d'utilisateurs à un tableau

— Gestion des tableaux
- ☑ Création, modification et suppression de tableaux
- ☑ Gestion des membres d'un tableau
- ☑ Création et suppression de colonnes

— Gestion des tâches
- ☑ Création, modification et suppression de tâches
- ☑ Attribution d'une tâche à un utilisateur
- ☑ Ajout d'une date limite et d'une priorité
- ☑ Ajout de commentaires

— Partie administration
- ☑ Gestion des comptes utilisateurs
- ☑ Statistiques globales :
  - — nombre de tableaux
  - — nombre de tâches
  - — activité des utilisateurs

— Options
- — IHM :
  - ☑ Réorganisation des colonnes
  - ☑ Déplacement de tâches entre colonnes (drag & drop)
  - ☑ Ajout de pièces jointes
- — Temps réel :
  - ☑ Mise à jour instantanée des tableaux avec WebSockets (par exemple lorsqu'une tâche est déplacée)
  - ☑ Notification lorsqu'une tâche est assignée ou commentée
- — Historique :
  - ☑ Journal des actions (création, déplacement et modification des tâches)
- — Recherche :
  - ☑ Recherche de tâches par mot-clé, utilisateur ou priorité

Toutes les demandes du cahier des charges ayant été réalisées dans les temps (que 2 jours perdus en somme pour la résolution de quelques fatal errors), nous avons pris la liberté de faire quelques ajouts :
- ☑ Statut tâche terminée, utilisé pour les filtres de recherche
- ☑ Ajout d'une page “Mes tâches”
- ☑ Modification du profil et du mot de passe
- ☑ Avatar (remplacé par initiales si absent)
- ☑ Notifications par email (inscription, invitation, nouveau commentaire)
- ☑ Export PDF du tableau (endpoint côté Spring)
- ☑ Statut en ligne des utilisateurs (petite icône verte sur l'avatar) via ws
- ☑ Conteneurisation Docker (nginx + Deno + Spring)
- ☑ Mot de passe oublié (avec token envoyé par email)

Problèmes et limites connus

Cette section recense les défauts identifiés mais non corrigés dans le temps imparti, ainsi que les décisions d'architecture qui nécessiteraient une refonte pour un passage en production.

⚠️ Déplacement de colonnes incompatible avec les WebSockets

Le réordonnancement des colonnes par drag & drop produit des résultats incohérents en présence de plusieurs clients connectés simultanément. Lorsqu'un utilisateur déplace une colonne, l'événement WS *column_moved* déclenche un *fetchColumns()* chez les autres clients, mais le numéro de colonne cible (*col_numero*) transmis dans le PATCH repose sur un snapshot pris au moment du *dragStart*. Si un autre client a modifié l'ordre entre-temps, les numéros ne correspondent plus et le résultat final diverge. La correction impliquerait de transmettre les identifiants de colonnes voisines plutôt qu'un numéro absolu.

⚠️ Réinitialisation du mot de passe sans jeton temporaire

La fonctionnalité "mot de passe oublié" modifie directement le mot de passe en base lors de la soumission du formulaire, sans étape de confirmation. Cette approche présente un risque : si un attaquant intercepte ou dévine le lien de réinitialisation, il peut changer le mot de passe de la victime sans que celle-ci l'ait demandé.

La correction consisterait à ajouter un champ *reset_token* (+ *reset_token_expiry*) dans la table *utilisateur_usr*. Lors de la demande, Deno génèrerait un token signé à durée limitée (ex. 15 min), stocké en base. Le lien envoyé par e-mail contiendrait ce token ; c'est seulement à sa présentation que le nouveau mot de passe serait effectivement appliqué à *usr_password* et le token invalidé.

⚠️ Le créateur d'une tâche est écrasé lors d'un déplacement

Lorsqu'un utilisateur déplace une tâche via drag & drop, la procédure stockée *deplacer_tache* reçoit le paramètre *usrAuteur* (l'utilisateur qui effectue l'action) et le stocke dans *tsk_createur*, écrasant ainsi le créateur original de la tâche. Ce bug provient d'une confusion entre "auteur du déplacement" (utilisé pour le journal d'activité) et "créateur de la tâche" (métadonnée immuable).

La correction nécessite une migration de schéma : ajouter un champ *tsk_auteur_derniere_modif* distinct de *tsk_createur*, puis modifier *deplacer_tache* pour n'écrire *usrAuteur* que dans le nouveau champ et dans *journal_log*, sans toucher à *tsk_createur*.

⚠️ Adresse hôte codée en dur dans les e-mails

Les liens envoyés par e-mail (réinitialisation de mot de passe, invitation) contiennent une URL dont l'hôte est codé en dur dans le code Deno. Cela oblige à modifier le source à chaque changement d'environnement (développement, serveur UBO, production) et empêche tout déploiement automatisé.

La correction consiste à récupérer dynamiquement le nom d'hôte depuis la requête entrante (*ctx.request.headers.get("host")*) ou depuis une variable d'environnement *APP_BASE_URL* déjà présente dans le *.env*, au même titre que *DB_HOST*.

⚠️ Aucun e-mail envoyé lors d'un retrait ou d'une suppression de compte

Lorsqu'un administrateur retire un membre d'un tableau, ou lorsqu'un compte utilisateur est supprimé (via la page d'administration), aucun e-mail de notification n'est envoyé à l'utilisateur concerné. Il perd l'accès sans en être informé, ce qui nuit à l'expérience utilisateur et peut créer de la confusion (sessions actives qui deviennent soudainement invalides).

La correction consiste à appeler le service d'envoi d'e-mail existant dans Deno aux deux points concernés : après *DELETE /boards/{id}/members/{userId}* et après *DELETE /users/{id}*.